Cybersecurity Hackathon Tech Stack Playbook

A decision-led field handbook for unknown problem statements, restricted ideation, and a working security MVP

Prepared for: HackMITTEN 3.0 and reusable cybersecurity hackathons
Planning context: HackMITTEN 3.0, 28-29 October 2026, MIT Thandavapura, Mysuru
Audience: Technical leads, builders, security/data owners, QA and demo owners
Purpose: Choose a defensible, buildable architecture quickly without relying on live internet access.

> **Operating rule:** One problem. One primary user workflow. One core analysis method. One reliable demo path. Prove the security outcome before polishing the dashboard.

How to use this playbook

This is a decision system, not a list of fashionable tools. Start with the quick reference, classify the problem, pick the closest architecture blueprint, then use the component matrices to replace any default that does not fit. During restricted ideation, follow the organizers' rules exactly. For HackMITTEN 3.0, the supplied participant instructions state that the first-round ideation prohibits mobile phones, laptops, electronic devices, AI tools, and external digital assistance. This document is preparation material; do not consult it during a restricted round unless organizers explicitly permit it. The event instructions describe a one-hour first round, so the 30-minute workflow below is a compact decision core that can fit inside that round, leaving the remaining time for the required case study/KG-sheet write-up and review.

Labels used

- **REQUIRED:** Needed to demonstrate the core outcome or meet a stated constraint.
- **DEFAULT:** Sensible starting point when no requirement contradicts it.
- **CONDITIONAL:** Add only when the problem needs it and the team can support it.
- **ADVANCED:** Usually avoid in a short hackathon unless it is central to the problem.

Contents

1. Executive quick-reference guide
2. Thirty-minute ideation decision workflow
3. Cybersecurity domain taxonomy
4. Technology stack catalog
5. Stack decision matrices
6. Ready-to-use architecture blueprints
7. Offline-first development strategy
8. Security product MVP design
9. Rules, ML, and LLM decision framework
10. Security, safety, and responsible testing
11. Team coordination and parallel development
12. Technical justification for judges
13. Failure modes and fallback plans
14. Testing and demonstration plan
15. Copy-ready decision templates
16. Pocket reference sheet
17. Technical accuracy and reference sources
18. Final technical-lead operating principles

1. Executive quick-reference guide

1.1 Stack choices to remember

Situation	Starting stack	Why it fits	Override when
DEFAULT: multi-page security product	React + TypeScript + Tailwind CSS + Vite; Python + FastAPI; SQLite; Python rules; pytest	Clear UI/API boundary, quick iteration, local persistence, explainable analysis	Team is unfamiliar with React, product is primarily analytical, or target platform imposes another stack
FAST: analytical prototype	Streamlit + Python + Pandas + SQLite; Plotly only if useful	Shortest path from files/data to inspectable findings	A highly customized multi-role interface or complex interaction is essential
OFFLINE-FIRST utility	Python CLI or Streamlit; SQLite/local files; preloaded fixtures	No cloud dependency; portable batch processing	A polished multi-page workflow is a judged core requirement
JavaScript-strong team	React + TypeScript + Node.js/Express + SQLite/PostgreSQL	One main language across UI and API; efficient parallel work	Python-only security/data libraries materially simplify the analysis
Data-intensive analysis	Python + Pandas/NumPy + SQLite; Streamlit; optional scikit-learn	Strong data wrangling and fast exploration	Large concurrent relational workloads or streaming ingestion are genuinely required
Live-event monitoring	React + FastAPI + SQLite/PostgreSQL; polling first, SSE if necessary	A live-looking queue can be reliable without complex streaming	True bidirectional interaction or high-throughput event delivery is required
Forensics/evidence reconstruction	Python + parsers + SQLite + local files; NetworkX optional	Reproducible parsing, timeline, relationship analysis, source references	Inputs are live enterprise telemetry rather than supplied evidence
No backend needed	Static UI, CLI, or Streamlit with local data	Less integration and failure surface	Multiple clients, permissions, durable API contracts, or shared records are needed

1.2 Minimum viable security architecture

```
Input (file, event, config, user action)
 -> validate and preserve source reference
 -> normalize into a small stable model
 -> analyze with one rule/model/policy engine
 -> rank findings with explainable reasons
 -> show evidence + uncertainty + recommended action
 -> record user decision/action and outcome
 -> test against known benign and suspicious fixtures
```

Do not add a separate database server, message broker, vector database, multi-agent framework, Kubernetes, or multiple microservices unless a stated requirement demands it.

1.3 Quick override rules

- Choose **SQLite** for a single-machine MVP and modest durable records. Choose PostgreSQL when concurrent users, relational queries, or stronger shared-database behavior matter. Choose MongoDB when document-shaped records and query patterns are a deliberate fit, not just because logs contain variable fields.

- Choose **rules first** when policy is known, outcomes must be explainable, and time is short. Choose ML only with relevant data and a baseline. Use an LLM for bounded explanation or summarization, never as the sole authority for a security verdict.
- Choose **polling** first for periodic refresh. Use Server-Sent Events (SSE) for mostly server-to-browser streaming; WebSockets for bidirectional live interaction.
- Choose **offline fixtures** as the primary demo path even when live feeds are available. Clearly label synthetic data and simulated actions.
- Choose **one working vertical slice** over multiple disconnected pages.

1.4 Commonly unnecessary in a short hackathon

Usually avoid advanced infrastructure, custom model training, live cloud feeds, external authentication dependencies, elaborate agent orchestration, complex event brokers, and production-scale claims unless required by the problem. Every dependency adds setup, debugging, security, and demo risk.

---

2. Thirty-minute ideation decision workflow

Use this compact workflow to reach architecture agreement in 30 minutes. If the event allots a full hour, use the remaining time to substantiate the case study, improve the written KG-sheet answer, examine edge cases, and ensure every team member can explain the chosen workflow. Do not browse during a device-restricted ideation phase.

Phase	Time	Ask and decide	Deliverable before moving on	Common mistake / fallback
A. Decode problem	0-4 min	Who is the user? What asset or outcome is at risk? What is explicitly required? What is out of scope?	One-sentence problem statement; explicit must-haves	Do not jump to a dashboard. If vague, state the safest testable interpretation and list assumptions.
B. Classify domain	4-7 min	Is this SOC, forensics, vulnerability, web/API, network, cloud, identity, privacy, education, GRC, or another family?	Primary domain + one secondary domain at most	Avoid calling every security tool a SOC. If mixed, classify by the core decision the user must make.
C. Inputs/outputs	7-10 min	Which files/events/configs /identities enter? What exact finding/action must emerge? What evidence justifies it?	Input, output, and success condition in plain language	Do not assume live feeds or labelled data exist. Use a supplied or synthetic fixture if allowed.

Phase	Time	Ask and decide	Deliverable before moving on	Common mistake / fallback
D. Workflow/architecture	10-14 min	What is the shortest useful path from input to defensible outcome? Does it need UI, API, persistence, live updates, or a model?	One end-to-end sequence and component diagram	Drop any component that cannot be justified in a sentence. Keep a simple single-process fallback.
E. Select stack	14-18 min	What are the team's strongest languages? Is offline operation required? Which database and analysis method fit?	One stack + one fallback stack	Do not choose tools by novelty. Prefer what is installed, familiar, and testable.
F. Define MVP	18-21 min	What is the one demo story? What is the minimum successful output? Which features are explicitly deferred?	3-5 MVP acceptance criteria; out-of-scope list	Avoid building every requested feature at once. If time is short, keep import, analyze, explain, persist, display.
G. Agree data contract	21-24 min	What fields does each component exchange? What are the request/response shapes and error cases?	Shared event/finding schema and endpoint names	Avoid parallel work against imaginary APIs. If uncertain, agree JSON examples first.
H. Assign owners and risks	24-27 min	Who owns UI, API/storage, analysis, tests/demo, and integration? What dependency could fail first?	Named ownership, first vertical slice, fallback	Do not wait until the end to integrate. Assign an integration owner even in a 3-person team.
I. Justify and freeze	27-30 min	Why this stack? What alternative did we reject? How do we prove correctness? What is the safe demo?	60-second pitch, test plan, frozen MVP	Do not make unverified claims. State limitations and label simulation.

Printable one-page decision checklist

- ☐ User, protected asset, security problem, and desired outcome are explicit.
- ☐ Primary domain is chosen; unnecessary domains/features are excluded.
- ☐ Inputs and outputs are named, with source/evidence references.
- ☐ One workflow is written from input to auditable outcome.
- ☐ One frontend/interface, one analysis engine, and only necessary backend/storage are selected.
- ☐ Chosen stack matches team skills and available/offline dependencies.

- ☐ A fallback exists for database, internet, API, model, and live data failure.
- ☐ A shared JSON contract and error shape are agreed.
- ☐ Each member owns a deliverable; one person owns integration.
- ☐ MVP has 3-5 testable acceptance criteria and an explicit out-of-scope list.
- ☐ Test fixtures include benign, suspicious, malformed, empty, and duplicate cases where relevant.
- ☐ Demo claims separate real capability, sample-data behavior, and simulation.
- ☐ We can justify the stack by requirements, trade-offs, security, and testing.

3. Cybersecurity domain taxonomy

The domain determines the analysis and evidence model; the interface is secondary. The defaults below assume a local prototype and may be substituted based on constraints. “Live data” means operational telemetry/feed access, not merely live UI refresh. Specialized hardware is generally unnecessary for a basic prototype unless the problem explicitly concerns it.

3.1 Domain decision table

Domain	Typical problem / user	Inputs -> core processing -> output	Architecture + default stack	Data, live/internet/hardware	Demo and common pitfall
1. SOC / incident response	Analyst prioritizes alerts and incidents	Logs/alerts -> normalize, deduplicate, correlate -> incident queue, evidence timeline, response status	Web UI + API + rules; React/FastAPI/SQLite or PostgreSQL	Events DB; live optional; internet not required; no special hardware	Replay labelled logs. Pitfall: duplicate alerts shown as separate incidents or unsupported “live” claims.
2. Threat intelligence / hunting	Analyst searches indicators and hypotheses	IOC lists + events -> normalize, enrich if available, match/correlate -> matches with source and timestamp	Python pipeline + small UI; FastAPI/SQLite	IOC/event store; feeds optional; internet optional; no hardware	Use local fixtures. Pitfall: an IOC match is presented as proof of compromise.
3. Vulnerability assessment/remediation	Developer/asset owner triages weaknesses	Scan/report/config -> deduplicate, map asset and severity, assign fix -> prioritized findings and verification status	Web workflow; React/FastAPI/SQLite or PostgreSQL	Relational model helpful; live scan optional; internet optional; no hardware	Use supplied scan results. Pitfall: claiming remediation without retesting.
4. Web application security	Developer tests a permitted app	Requests, code, test results -> validation/static rules/safe test cases -> finding and reproduction evidence	API/CLI + report UI; Python or Node; pytest/httpx	Files/results DB; live app not required for fixture demo; internet not required	Use local labs/fixtures. Pitfall: scanning targets without explicit authorization.

Domain	Typical problem / user	Inputs -> core processing -> output	Architecture + default stack	Data, live/internet/hardware	Demo and common pitfall
5. API security	API owner checks access, schema, abuse controls	OpenAPI/spec, requests, responses -> policy/schema checks and authorized tests -> endpoint risk report	FastAPI/Node utility + tests; React optional	API inventory/results DB; live service optional; internet optional	Test local/supplied API. Pitfall: testing only authentication while missing object/function authorization.
6. Network security / IDS	Network analyst investigates traffic	PCAP, flows, DNS/firewall logs -> parse, extract features, rules -> suspicious flows and packet/event refs	Python + Scapy if installed + Pandas; CLI/Streamlit	Local files/SQLite; live capture optional; internet not required; packet capture permissions may matter	Use prebuilt PCAP/flow rows. Pitfall: requiring live capture or privileged interfaces to make the demo work.
7. Endpoint security	Admin monitors endpoint events	Process/file/auth events -> rules, baselines, correlation -> endpoint alerts and action history	Local agent simulation + API/UI; Python/FastAPI/SQLite	Event DB; live agent optional; internet optional; admin privileges only if real collection needs them	Simulate endpoint events. Pitfall: implying OS-wide detection from a sample feed.
8. Ransomware resilience / file integrity	Owner detects unexpected file changes	Safe test files + metadata/hashes -> baseline comparison and change grouping -> changed-file report and recovery checklist	Python utility + SQLite; optional Streamlit	Local files; live monitoring optional; internet unnecessary; no special hardware	Use generated test directory only. Pitfall: destructive samples or modifying user files.
9. Digital forensics / reconstruction	Investigator builds a timeline	Logs, metadata, supplied artifacts -> parse, normalize timezone, correlate entities -> timeline, lineage, evidence report	Offline Python + SQLite + Pandas; NetworkX optional	Local evidence store; no live data/internet needed; no hardware for file-based cases	Use immutable sample fixture. Pitfall: losing original references or silently rewriting timestamps.

Domain	Typical problem / user	Inputs -> core processing -> output	Architecture + default stack	Data, live/internet/hardware	Demo and common pitfall
10. Zero Trust / identity & access	Admin checks who can access what and why	Users, devices, resources, policies, context -> policy evaluation -> allow/deny with reason and audit trail	Web/API + policy engine; React/FastAPI/SQLite/PostgreSQL	Relational storage helpful; live IdP optional; internet optional	Use synthetic identities. Pitfall: plaintext passwords or “Zero Trust” branding without explicit policy evaluation.
11. Cloud security	Cloud/security engineer checks resource posture	Config snapshots, IAM/network policies -> rule checks and resource mapping -> misconfiguration findings and fixes	Python config parser + UI; Streamlit/React + SQLite	Snapshot files; live account unnecessary; internet optional; no special hardware	Parse supplied JSON/YAML. Pitfall: requiring cloud credentials for a snapshot-based problem.
12. Container / supply-chain security	Developer identifies risky build/dependencies	SBOM, lockfiles, image metadata, config -> package/policy checks -> dependency findings and action list	CLI/CI-style pipeline; Python/Node; local JSON outputs	File artifacts/results DB optional; live registry optional; internet not needed	Use a supplied SBOM/lockfile. Pitfall: making an unverified claim that an image is safe.
13. DevSecOps / secure SDLC	Team adds checks to development workflow	Source, commit, CI output, rules -> static/policy checks -> developer findings, gate status	CLI/API + report; existing language tools + pytest	File/report storage; live CI not required; internet optional	Run on demo repository/fixtures. Pitfall: building a full CI platform instead of one actionable check.
14. Source-code analysis	Developer finds risky patterns	Source files -> parsers/regex/AST/rules -> source-linked findings and safer alternatives	CLI or Streamlit; Python parser/rules	Local files; no live data/internet required	Small known fixture. Pitfall: regex-only results described as complete semantic analysis.
15. Privacy / leakage prevention	Privacy officer reviews sensitive data	CSV/docs/logs -> detect patterns, classify, redact -> review queue and sanitized export	Python + structured parsers/regex + Streamlit/FastAPI	Local storage; no external APIs by default; internet unnecessary	Synthetic PII fixture. Pitfall: uploading sensitive data to an external LLM or overclaiming perfect detection.

Domain	Typical problem / user	Inputs -> core processing -> output	Architecture + default stack	Data, live/internet/hardware	Demo and common pitfall
16. Cryptography / key management	Engineer validates usage or workflow	Config/key metadata/test vectors -> policy checks and crypto-library operations -> validation result and audit event	Small service/CLI; language's standard crypto libraries	Avoid storing secrets in demo; live services optional; no hardware unless HSM requirement explicit	Show safe test vectors. Pitfall: inventing cryptography or hardcoding production secrets.
17. Fraud / anomalous activity	Analyst reviews account/transaction patterns	Events, transaction records -> rules/statistics/baseline -> flagged cases with reasons	Python/Pandas + Streamlit/API; SQLite	Relational/event store; live feed optional; no hardware	Synthetic labelled events. Pitfall: treating anomaly as fraud without investigation or calibration.
18. AI/ML-assisted cybersecurity	Analyst needs ranking or pattern detection	Labelled or unlabeled events -> baseline + measured model -> score plus evidence and uncertainty	Python/Pandas/scikit-learn + Streamlit/FastAPI	Dataset is key; internet/LLM optional; no hardware for small tabular models	Compare with baseline. Pitfall: training with insufficient data or inventing accuracy.
19. Cybersecurity education/awareness	Teacher/student practices safe decisions	Scenarios, quiz responses -> rubric/feedback -> explanation, learning progress	React or Streamlit; backend optional; SQLite for saved progress	Local scenario data; internet optional; no special hardware	Scenario-based walkthrough. Pitfall: score alone treated as proof of practical competence.
20. GRC / audit / compliance	Auditor maps evidence to controls	Policy, control checklist, evidence -> status/risk/gap mapping -> evidence-linked report and owner/actions	Web form/report; SQLite/PostgreSQL; rules	Structured records; live data not required; internet optional for standards lookup	Use sample control set. Pitfall: presenting a prototype as certification or legal advice.
21. Digital evidence management	Investigator needs chain-of-custody records	Evidence files/metadata -> ingest, hash, custody events -> manifest and access/change history	Offline utility + API; Python/SQLite/local files	Local files/DB; live data no; internet no; storage integrity matters	Hash and track supplied files. Pitfall: confusing a hash with proof of provenance/authentic collection.

Domain	Typical problem / user	Inputs -> core processing -> output	Architecture + default stack	Data, live/internet/hardware	Demo and common pitfall
22. Security automation / orchestration	Analyst automates repetitive approved actions	Findings, policy, approval -> workflow/state machine -> simulated or approved action, audit outcome	FastAPI + task/state workflow; queue only if needed	DB for tasks/audit; live integrations optional; internet optional	Simulate containment. Pitfall: dangerous autonomous actions without human approval or rollback.
23. IoT / embedded security	Engineer checks device posture or telemetry	Device config, firmware metadata, sensor events -> policy/protocol parsing -> risk findings	Python parser/simulator + UI	Snapshot or device fixtures; live hardware optional; special hardware only if problem needs it	Use virtual devices/fixtures. Pitfall: promising firmware testing without actual analysis capability.
24. Security analytics / risk intelligence	Leader wants risk trends and priorities	Findings, assets, control results -> normalize, aggregate, trend -> explainable risk views and actions	SQL/Pandas + UI; SQLite/PostgreSQL	Historical records needed for trends; live data optional; no hardware	Use reproducible sample data. Pitfall: attractive charts with undefined risk formulas or misleading score precision.

3.2 Fast domain selection questions

- Is the main output a **decision about events**? Start with SOC/analytics and event correlation.
- Is the main output a **reconstructed story from existing evidence**? Start with forensics/evidence management.
- Is the main output a **fixable weakness attached to an asset**? Start with vulnerability/configuration/source analysis.
- Is the main output **allow/deny with a policy reason**? Start with identity/Zero Trust.
- Is the main output a **learning intervention**? Start with education, not a security operations stack.
- Is the main output a **measured prediction**? Check dataset quality and baseline before choosing ML.

4. Technology stack catalog

4.1 Frontend / interface

Technology	Purpose and good fit	Advantages	Limitations and offline/team notes	Choose / avoid
React	Custom, multi-page interactive product	Reusable components; flexible UI; broad ecosystem	Build/configuration and API integration take time; local packages must already be installed for offline work	Choose for judged multi-page workflows; avoid if a chart/form is the whole product and the team is unfamiliar

Technology	Purpose and good fit	Advantages	Limitations and offline/team notes	Choose / avoid
TypeScript	Typed JavaScript for UI/API contracts	Catches many shape/type mismatches earlier; improves refactoring	Adds type definitions and build errors to learn	Default for a React team comfortable with it; plain JS is acceptable if it saves time without losing contract clarity
Tailwind CSS	Utility-first styling	Fast consistent styling; easy responsive layout	Utility-heavy markup and build setup can distract from function	Choose when already familiar; avoid redesigning every screen before analysis works
Vite	Frontend dev/build tool	Fast local development workflow	Requires dependency setup and build knowledge	Strong default for a standalone React prototype; not itself a UI framework
Next.js	React framework with routing/server rendering options	Useful for product needs that require its conventions	Additional routing/runtime concepts and possible deployment complexity	Choose only when its rendering/routing capabilities are required or team already knows it
Streamlit	Python-first analytical app	Very fast file upload, tables, charts, filters	Less control over highly custom multi-page UX and interaction patterns	Choose for security-data exploration; avoid for complex role-based product workflows unless sufficient
Gradio	Quick UI around model or function inputs/outputs	Fast demo of inference or processing	Not a complete product architecture by itself	Choose for an ML/model interface; avoid when a multi-role workflow is the core product
Plain HTML/CSS/JS	Minimal browser UI	Small setup and no framework overhead	More manual state/component management	Choose for very small, static, single-flow demos; avoid creating an ad-hoc framework for a larger UI
CLI	Command-line interface	Lowest UI cost; reproducible batch processing	Poor fit if judges/users need guided visual workflow	Choose when the core claim is parsing/analysis; add a minimal report only if valuable

Frontend rule: Pick the least complex interface that makes the core outcome inspectable. A “professional” UI is one where users can understand the result and evidence, not merely one with animation and charts.

4.2 Backend / API

Technology	Purpose	Advantages	Limitations / selection rule
Python	Programming language for analysis, APIs, parsing, automation	Security/data ecosystem; quick scripting	Runtime and package versions still need management; language itself is not a web framework
FastAPI	Python API framework	Typed request/response models, automatic API schema, async support where applicable	A separate frontend and backend create integration work; do not use async unless workload/library requires it
Flask	Lightweight Python web framework	Simple, flexible, low conceptual overhead	More design choices and conventions are left to the team; useful if already familiar
Django	Full web framework	Integrated ORM, admin, auth and common web patterns	More framework than a narrow analysis MVP may need; setup and concepts can slow a short build
Node.js + Express	JavaScript runtime + minimal server framework	One language across React and server; straightforward APIs	Python analysis libraries may require a second service/process if used; Express is not a programming language
No separate backend	Single-process Streamlit, CLI, or static tool	Fewest moving parts	Not suited to multiple clients, server-side access control, or shared API contracts

Important distinction: Python and JavaScript are languages/runtimes. FastAPI, Flask, Django, and Express are frameworks used to build applications or APIs. React is a UI library; Vite is a development/build tool. Do not mix these categories in a judge pitch.

4.3 Database and storage

Option	Best fit	Benefits	Limitations / risks	Offline / concurrency
SQLite	Single-machine MVP, cases/findings, small relational dataset	File-backed, low setup, SQL and transactions	Not a central multi-server database; concurrent write workload has limits	Excellent offline choice; supports multiple readers and controlled write concurrency
PostgreSQL	Shared application with relationships, joins, constraints, concurrency	Strong relational features and query capabilities	Separate service/setup and credentials need time	Works locally; add when relationships, concurrent users, or deployment justify it

Option	Best fit	Benefits	Limitations / risks	Offline / concurrency
MongoDB	Document-shaped records and access patterns suited to documents	Flexible document model and nested fields	Separate service and operational configuration; flexibility can weaken consistency if schemas are ungoverned	Can run locally if installed; don't choose solely because events have variable attributes
In-memory dictionaries	Temporary computation or disposable demo state	Almost no setup	Lost on restart; no audit/history guarantee	Offline, but not persistent; state clearly if used only for a throwaway proof
Local JSON/CSV	Fixtures, imports/exports, small append-only outputs	Human-readable and simple	Weak concurrency/constraints; awkward joins and updates	Strong offline fixture and interchange formats
Local file/object storage	Original evidence, uploaded artifacts, bulky reports	Keeps binary evidence outside relational rows	Requires safe paths, access checks, size limits, cleanup, hashes, and metadata references	Local path works offline; maintain references and a custody/audit record

Storage decision: model entities and relationships first; then select storage. Use normalized relational tables when referential integrity, joins, and audit history matter. Use document storage when records are naturally self-contained and queries match that structure. Preserve raw events or evidence references where practical. Define retention, maximum upload size, pagination, duplicate handling, and restart behavior.

4.4 Security analysis and detection

Method/tool	Use when	Good practice and limitations
Native Python rules	The policy or suspicious pattern can be written explicitly	Include rule ID/version, rationale, inputs, tests, severity logic, and known blind spots
Pandas / NumPy	Tabular logs, CSVs, timestamps, aggregation, numerical features	Validate columns and types; handle missing values/timezones; avoid loading huge files unboundedly
scikit-learn	A measured statistical or supervised/unsupervised baseline is useful	Define labels, leakage-safe split, baseline, metrics, and false-positive review; no accuracy claim without evaluation
NetworkX	Entity relationships, event lineage, graph paths	Graphs can clarify links but do not prove causality; retain edge evidence and timestamps
Scapy	Parsing/working with packet data when installed and authorized	Packet capture may need privileges and environment support; use supplied PCAP/flow fixtures as fallback
Log parsing	Structured JSON/CSV first; regex for bounded unstructured formats	Preserve source offsets/IDs; reject malformed rows safely; don't silently drop data

Method/tool	Use when	Good practice and limitations
Static code analysis	Known source patterns or AST-based checks	Report source locations, rule rationale, and severity; distinguish heuristic findings from proven exploitability
Configuration policy checks	Cloud, container, identity, and hardening snapshots	State exactly which policy was checked and which data was absent; no credentials required for snapshot mode
File-integrity analysis	Baseline-vs-current comparisons and evidence handling	Hashes help detect change; do not claim a hash alone proves authenticity or provenance
IOC matching	Known indicators compared against supplied events	Normalize carefully and preserve source/date; a match is a lead, not proof of compromise
Event correlation	Related events grouped by time, asset, account, IP, process, or case	Use explainable join keys/windows, deduplicate, store underlying event references, and handle timezones
Risk scoring	Prioritize based on severity, exposure, confidence, business context	Show formula and factors; avoid false precision and calibrated-probability language unless actually calibrated
LLM-assisted interpretation	Draft summaries, translate technical findings, answer bounded evidence questions	Retrieve relevant evidence first, cite event IDs, label unknowns, prevent data leakage, and keep a deterministic path if API fails

Rules vs statistics vs ML vs generative AI

- **Deterministic rules:** explicit conditions/policy; best for known patterns, explainability, quick MVPs, and access decisions.
- **Statistical anomaly detection:** estimates deviation from a baseline; useful where normal behavior can be described and anomalies merit review, but anomaly is not maliciousness.
- **Supervised ML:** learns labelled classes; requires representative labels and split discipline. Measure precision, recall, false positives/negatives, and baseline comparison.
- **Unsupervised ML:** groups or scores without labels; useful for exploration, but interpretation and validation remain hard.
- **Generative AI / LLM:** language generation and synthesis; useful for summaries and question answering over retrieved evidence, but may hallucinate. It is not inherently a superior detector.

4.5 Real-time communication

Method	Choose for	Cost / failure fallback
Normal HTTP request	Import, analyze, search, save, and retrieve workflows	Simplest; show loading/error states and retry safe operations carefully
Polling	Dashboard can refresh every few seconds or on demand	Easy to implement; cap frequency and avoid duplicate loads
Server-Sent Events (SSE)	Server-to-browser event updates where client doesn't need a bidirectional stream	Simpler than WebSockets for one-way updates; fall back to polling
WebSockets	Bidirectional interactive live session, commands, chat, or fast two-way events	More lifecycle/reconnect logic; use only if requirement warrants it

Method	Choose for	Cost / failure fallback
Message queue	Independent producers/consumers, buffered workload, durable asynchronous tasks	Operational complexity; usually unnecessary for a single-machine MVP

A UI that polls local synthetic events can demonstrate the workflow but must be called a replay/simulation, not production telemetry monitoring.

4.6 Testing and quality

- **Unit:** pytest for Python rules/parsers; test positive, negative, boundary, malformed, and empty cases.
- **API:** FastAPI test client or HTTPX-based tests; verify status codes, schema validation, authorization where applicable, and safe errors.
- **Frontend:** use the team's existing test tool; prioritize form validation, empty/loading/error states, and correct display of evidence/reasons.
- **Integration:** import -> analyze -> persistence -> retrieval -> action history. Use a temporary local database or isolated fixture.
- **End-to-end:** rehearse the exact judging path from clean startup to visible outcome.
- **Security regression:** unauthorized access, input limits, file upload restrictions, secret handling, and safe simulation boundaries.
- **Performance:** measure actual bottlenecks if required; use a simple representative fixture first. Avoid claiming capacity from a tiny dataset.
- **Test data:** keep known expected outcomes beside each fixture. Label synthetic data and do not disguise it as live telemetry.

4.7 Development, deployment and reproducibility

- Use a project virtual environment and a pinned or recorded dependency set; do not depend on internet installs during the demo.
- Keep secrets in environment variables or local ignored configuration; commit a `.env.example`, never actual credentials.
- Maintain a README with prerequisites, launch commands, fixture locations, ports, database initialization, and reset instructions.
- Add structured logs with severity, timestamp, request/incident ID, and error category. Avoid logging secrets or full sensitive payloads.
- Handle errors at boundaries; show useful user-facing messages without tracebacks or internal paths.
- Make database setup and test data loading repeatable. Define a safe lab reset that clears only expected demo data.
- Docker is conditional: useful for reproducibility if already familiar and installed, but not mandatory for a local prototype.
- Use Git branches/commits sensibly; integrate early and keep one known-good launch path.

5. Stack decision matrices

Ratings are qualitative and contextual, not universal benchmarks. "High" can mean capability or setup cost as indicated. Team familiarity may dominate theoretical flexibility.

5.1 Frontend selection

Option	Speed to first demo	Custom UX	Analysis/data UI	Setup cost	Good default
CLI	Very high	Low	Medium	Very low	Batch processing / parser proof

Option	Speed to first demo	Custom UX	Analysis/data UI	Setup cost	Good default
Streamlit	High	Medium-low	High	Low	Data and investigation prototype
React + TypeScript	Medium	High	High	Medium	Multi-page security product
Plain HTML/CSS/JS	High for tiny app	Medium	Low-medium	Low	Single small workflow
Next.js	Medium	High	High	Medium-high	Only when its framework features are needed
Gradio	High for model I/O	Low-medium	Medium	Low	Model demo

5.2 Backend selection

Option	Speed	Integrated conventions	Python analysis integration	Best context
No separate backend	Very high	Low	Excellent if Python UI/app	CLI or Streamlit single-process MVP
FastAPI	High	Medium	Excellent	Typed Python API, validation, docs
Flask	High	Low-medium	Excellent	Small app and familiar team
Django	Medium	High	Good	Full web app with built-in ORM/auth/admin needs
Node + Express	High for JS team	Low-medium	Requires bridge if Python analysis central	Team strongest in JavaScript

5.3 Database selection

Option	Durable	Relationships/ joins	Flexible documents	Setup	Recommendation
In-memory	No	No	Any transient object	Very low	Temporary proof only
JSON/CSV	Yes as files	Weak/manual	File-shaped	Very low	Fixtures/ import/export, not multi-user case management
SQLite	Yes	Strong SQL for local data	Limited via JSON support and schema choices	Low	Default local MVP
PostgreSQL	Yes	Strong	Can support semi-structured fields too	Medium	Shared/ concurrent relational app

Option	Durable	Relationships/ joins	Flexible documents	Setup	Recommendation
MongoDB	Yes	Different model/query patterns	Strong document fit	Medium	Select for document-first access patterns, not as a reflex

5.4 Analysis engine selection

Need	Start with	Add conditionally	Evidence to prove it works
Known policy	Deterministic rules	Rule configuration or policy language	Unit tests + rule rationale
Tabular log analytics	Pandas/SQL	NumPy/visualization	Expected counts and fixture comparisons
Relationship/timeline analysis	Normalized events + time-window rules	NetworkX	Evidence-linked edges and timeline tests
Packet features	Supplied flows / PCAP parser	Scapy	Known packet/flow fixture results
Source/config checking	Explicit rules and parsers	AST or specialized scanners	Source/config location and test cases
Predictive classification	Baseline + labeled dataset	scikit-learn model	Precision/recall, confusion matrix, held-out set
Natural-language explanation	Evidence retrieval + templates	LLM API/local model if permitted	Grounding checks, citations to event IDs, failure fallback

5.5 Real-time architecture selection

Requirement	Preferred	Avoid when
User clicks to refresh	HTTP	Never needed to add streaming
Updates every few seconds	Polling	Update frequency is genuinely too high or wasteful
Server pushes one-way updates	SSE	Bidirectional client commands are required
Two-way live session	WebSockets	Periodic one-way status is enough
Durable decoupled background jobs	Queue/task worker	Single-process job can finish in request time and reliability is adequate

5.6 Deployment selection

Option	Works best when	Primary risk
Local process / script	One laptop, offline, small MVP	Environment drift and port conflicts
Local web app	UI and backend run on same machine	Startup sequencing and CORS/API URL mismatch
Docker Compose	Dependencies are known and Docker is already available	Image build/pull/network assumptions; setup overhead
Hosted service	A reachable internet and deployment permissions are assured	Connectivity, credentials, quotas, and last-minute deploy failures

5.7 AI/ML selection

Evidence and requirement	Decision	Why
Explicit policy / known pattern	Rules	Fast, explainable, testable
No labeled data, but useful statistical baseline	Descriptive statistics/anomaly ranking	Can surface outliers without pretending to know maliciousness
Representative labels and measurable value over baseline	Supervised ML experiment	Data and evaluation support model selection
Large unlabeled dataset for exploration	Unsupervised method, cautiously	Useful for grouping but not self-validating
Need to summarize already-grounded findings	Templates first; optional LLM	Language assistance is separate from detection correctness
No dataset, no evaluation, no clear model-specific need	No model	Model would add risk without evidence of benefit

5.8 Offline vs connected architecture

Capability	Offline-first	Connected / live option	Fallback rule
Threat intelligence	Bundled dated feed/sample	Update feed when reachable	Use local feed and show its timestamp/source
AI interpretation	Templates or local model if installed/allowed	External LLM API	Core findings must remain available without it
Authentication	Local demo roles or approved test identity	Enterprise IdP	Do not put the IdP on the only demo path
Data	SQLite and local files	Managed DB/object store	Export/fixture-based demonstration must still work
Monitoring	Replay/synthetic event generator	Endpoint/cloud/network telemetry	Label replay and never describe it as live collection
Dependencies	Preinstalled environment/lock file/wheel cache when permitted	Package registry	Follow event rules; do not assume downloads or preloaded code are allowed

6. Ready-to-use architecture blueprints

Each blueprint is a starting layout, not a template to force onto every problem. Keep components inside one process unless separation solves a specific requirement.

Blueprint A — Professional multi-page cybersecurity web application

- **Problem fit:** multi-role findings, cases, policies, or asset workflows requiring clear navigation and durable records.
- **Architecture:** React/TypeScript UI -> FastAPI API -> domain services/rules -> SQLite initially; PostgreSQL if relational concurrency needs justify it.
- **Component responsibilities:** UI owns workflow presentation and validation; API owns schema/auth boundary; domain service performs analysis; storage stores findings and action history; tests verify the vertical slice.
- **Data flow:** user uploads/selects a fixture -> API validates -> service analyzes -> finding persisted -> UI shows evidence, rationale, and status -> action recorded.

- **MVP:** overview/status, list/search, finding detail, evidence references, one action/status update, test and reset path.
- **Optional:** roles, advanced filters, SSE, export reports.
- **Setup/offline:** frontend build tool and Python environment preinstalled; local DB and fixture; one documented launch method.
- **Risks/tests:** CORS/port mismatch, contract drift, unbounded uploads, mock data never replaced. Test import-to-action and restart persistence.
- **Switch when:** analytical exploration is the product's core value and custom UX adds little -> Streamlit; team is strongly JS-oriented -> Node/Express.

Blueprint B — Fast data-analysis / security-intelligence prototype

- **Problem fit:** CSV/JSON log triage, risk scoring, indicators, trends, or exploratory analytics.
- **Stack:** Python + Pandas + SQLite + Streamlit; visualization library optional.
- **Flow:** upload local file -> validate required columns -> normalize -> compute rules/aggregates -> show finding table and charts -> export report.
- **Modules:** import validation, schema mapping, analysis rules, persistence/cache, result explanation.
- **MVP:** supported input schema, errors for malformed files, 3-5 core rules/metrics, evidence row/source, export, known fixture tests.
- **Optional:** ML baseline, graph relationships, reusable parsers.
- **Risks/tests:** large file memory usage, timezone inconsistencies, missing columns, seductive but unexplained charts. Test empty/invalid/duplicate input and expected totals.
- **Switch when:** custom multi-page interactions/roles are central -> Blueprint A; command-line batch processing is sufficient -> CLI.

Blueprint C — Offline digital-forensics application

- **Problem fit:** evidence import, event timeline, entity relationships, source-linked reconstruction.
- **Stack:** Python parsers + Pandas when tabular + SQLite + local evidence directory; Streamlit or a CLI; NetworkX only when graph queries matter.
- **Flow:** ingest copy/reference -> record metadata and SHA-256 hash -> parse without changing original -> normalize timestamps with source timezone metadata -> correlate -> report timeline/findings with source references.
- **MVP:** sample evidence import, parser status, timeline sort, entity links, hash manifest, report export, reproducible test.
- **Optional:** broader file formats, signed manifests, deletion recovery research module if safe and in scope.
- **Setup/offline:** all sample files and packages local; no external APIs.
- **Risks/tests:** timezone drift, parser errors hidden, source file overwritten, hash presented as authenticity proof. Preserve originals, test known timestamps, record errors and collection limits.
- **Switch when:** live continuous telemetry and response workflows are central -> Blueprint D.

Blueprint D — SOC dashboard with incident correlation and response tracking

- **Problem fit:** alert triage, incident grouping, evidence timeline, response workflow.
- **Stack:** React + FastAPI + SQLite/PostgreSQL; rule-based normalizer/correlator; polling first; SSE conditional.
- **Flow:** fixture/log import or authorized source -> validate/deduplicate -> normalize -> correlate by asset/account/time/indicator -> priority and rationale -> incident queue -> analyst action -> audit history.
- **Minimum schema:** event, finding, incident, evidence reference, action/audit entry, rule version.
- **MVP:** replay labelled events; group related alerts; show supporting events and reason; status transition; one recommended safe response; persistence and dedupe test.
- **Optional:** feed connectors, role controls, streaming, playbooks after approval.
- **Risks/tests:** one event becomes many logs, stale counts, false correlation, alert loops, simulated actions mistaken for real containment. Test duplicates, correlation windows, clean reset and restart.
- **Switch when:** core task is only CSV-based analytical reporting -> Blueprint B.

Blueprint E — Vulnerability management and remediation

- **Problem fit:** consolidate scan results, map findings to assets, assign remediation, verify closure.
- **Stack:** React/Streamlit + FastAPI + SQLite/PostgreSQL; parsers and deterministic prioritization.
- **Flow:** import authorized scan/snapshot -> normalize asset/CVE/config rule -> deduplicate -> rank by severity/exposure/criticality with disclosed formula -> assign owner -> retest/mark verified.
- **MVP:** asset list, findings import, rationale, owner/status, verification field, sample report.
- **Optional:** CVE feed refresh, asset graph, change history.
- **Risks/tests:** duplicate identifiers, missing versions, stale external feeds, “fixed” status with no proof. Test import duplicates and explicit verified/unverified status.
- **Switch when:** no host scanning is allowed and only configuration snapshots exist -> configuration analyzer mode.

Blueprint F — Zero Trust / identity-policy demonstrator

- **Problem fit:** show least-privilege decisions and why access is granted or denied.
- **Stack:** React + FastAPI + SQLite/PostgreSQL; pure policy evaluation function.
- **Flow:** synthetic principal + device/context + resource + policy -> evaluate -> allow/deny + matched rule/reason -> write audit entry.
- **MVP:** user/resource/policy examples, deny-by-default scenario, policy explanation, audit history, tests for allowed and forbidden combinations.
- **Optional:** role-based policy editor, simulated risk context.
- **Risks/tests:** hard-coded decision paths inconsistent with UI, demo passwords, vague “Zero Trust” claims. Test least-privilege boundaries and no plaintext secrets.
- **Switch when:** policy evidence comes from real identity provider -> add connector only if permitted/available.

Blueprint G — Network monitoring / intrusion detection

- **Problem fit:** inspect provided traffic records and identify suspicious patterns.
- **Stack:** Python + flow/PCAP parser; Scapy if available; Pandas + SQLite; Streamlit or CLI.
- **Flow:** PCAP/flow import -> parse safely -> calculate features -> apply explicit rules -> display matching packet/flow references -> report.
- **MVP:** preloaded PCAP or CSV flows, 2-4 transparent rules, benign/suspicious fixtures, rule evidence, limitations note.
- **Optional:** authorized live capture, enrichment if connected.
- **Risks/tests:** permissions, interface mismatch, empty capture, rate/volume issues. Test against known supplied fixture and offer prepared flows if live capture fails.
- **Switch when:** input is endpoint logs or cloud telemetry rather than network traffic.

Blueprint H — Cloud-configuration / compliance analyzer

- **Problem fit:** inspect configuration snapshots against explicit security rules.
- **Stack:** Python + JSON/YAML parsers + rule definitions + SQLite + Streamlit/React.
- **Flow:** import snapshot -> schema validation -> map resource type -> evaluate policy -> finding with field path and remediation -> export.
- **MVP:** 2-3 config types, explicit rules, resource path, severity and fix guidance, passing and failing fixtures.
- **Optional:** policy pack, cloud connector, control mapping.
- **Risks/tests:** unsupported config version, missing field interpreted as safe, overbroad compliance claim. Mark unknown/not-applicable separately from pass.
- **Switch when:** actual live account collection is required and authorized; only add SDKs/credentials if available.

Blueprint I — AI/ML-assisted cybersecurity tool

- **Problem fit:** classification or ranking where historical data supports an evaluated model, or language assistance on already-grounded findings.

- **Stack:** Python + Pandas; scikit-learn only with a baseline; Streamlit/FastAPI; local fixture; optional LLM adapter.
- **Flow:** validate dataset -> split safely -> baseline -> model -> evaluate -> emit score with explanation/limits -> optionally summarize evidence.
- **MVP:** baseline comparison, reproducible evaluation, test set, metric table, confidence/uncertainty note, non-AI fallback.
- **Optional:** model registry/versioning and drift tests outside hackathon scope.
- **Risks/tests:** leakage, tiny labels, uncalibrated scores, generated evidence, API failure. Test prompt injection in untrusted data if LLM is used, and verify every claim links to evidence.
- **Switch when:** dataset does not support evaluation -> deterministic rules/statistical summary.

Blueprint J — Cybersecurity education / awareness

- **Problem fit:** interactive scenarios, knowledge checks, explanations, teacher/student insight.
- **Stack:** React or Streamlit; lightweight backend optional; SQLite if progress needs persistence.
- **Flow:** select scenario -> present realistic but safe choice -> score with explicit rubric -> explain why -> record progress -> educator sees aggregated trends.
- **MVP:** 3-5 scenario types, deterministic scoring, individual result, rationale, teacher summary using synthetic students.
- **Optional:** class-level analytics, authoring, localized language, LLM feedback with constrained rubric.
- **Risks/tests:** score treated as real-world competence, student privacy, teacher/student access boundary, model feedback drift. Test answer keys, role isolation, and saved-progress behavior.
- **Switch when:** primary goal becomes policy compliance or audit evidence rather than learning.

---

7. Offline-first development strategy

Offline-first means the **core value still works** with no internet or external services. It does not mean a team can ignore event rules about what devices, code, notes, or datasets may be brought in.

7.1 Design the primary path locally

- Use supplied or synthetic fixtures. Label them as synthetic, simulated, replayed, or real according to their provenance.
- Store demo records in SQLite or local files. Keep original artifacts separate from normalized rows.
- Record source filenames, timestamps, parser version, rule version, and evidence references.
- Include known benign and suspicious test cases where labels are available.
- Record dependencies and verify they are installed when permitted. Do not assume package downloads will work.
- Include short local startup instructions and a deterministic demo dataset.
- If using an optional external API, fail gracefully and preserve core findings without it.
- Keep local documentation for schema, run steps, fixture assumptions, and known limitations.
- Make reset behavior explicit and safe: only reset the demo data directory/database, never broad user paths.
- Follow rules regarding prewritten notes, devices, code, and datasets; do not assume permission.

7.2 Offline ideation vs offline implementation

Phase	Offline requirement	Recommended behavior
Restricted ideation	Devices and AI may be explicitly prohibited	Reason from memory and paper; use only materials allowed by organizers
Coding	Internet may be absent or limited	Use installed dependencies, local fixtures, documented commands, and a local database

Phase	Offline requirement	Recommended behavior
Testing	External services may be unreachable	Use deterministic fixtures and explicit expected outputs
Demo	Judge network may fail	Lead with offline fixture; live integrations remain optional and visibly labelled
Post-hackathon	Research/update work may be allowed	Verify current versions, patch dependencies, improve security and test coverage

7.3 Offline failure fallbacks

- Threat feed unavailable -> use a bundled, dated fixture and show source timestamp.
- LLM unavailable -> use a template explanation based on structured fields.
- DB unavailable -> attempt an approved local SQLite file or read-only fixture; do not silently drop persistence claims.
- Package missing -> remove the optional feature or use a simpler standard-library path; do not make uncontrolled installs a critical dependency.
- Live capture blocked -> load a prepared PCAP/flow file and disclose replay mode.
- External identity provider unavailable -> use clearly labelled synthetic users and a local policy demonstration.

8. Security product MVP design

8.1 General pipeline

```
INPUT -> VALIDATION -> NORMALIZATION -> SECURITY ANALYSIS
   -> CORRELATION / PRIORITIZATION -> EVIDENCE + EXPLANATION
   -> RECOMMENDED ACTION -> AUDITABLE OUTCOME
```

Remove stages that do not fit the domain. For example, a learning scenario may need scoring and feedback, not an incident correlation engine. A configuration checker may not require live data or an LLM.

8.2 Minimum data model

Event (for event-centric projects):

```
{
 "event_id": "evt-001",
 "timestamp": "2026-10-28T10:25:00Z",
 "source": "sample-firewall.csv:row-18",
 "asset": "demo-host-01",
 "actor": "demo-user",
 "event_type": "authentication_failure",
 "attributes": {"count": 6},
 "synthetic": true
}
```

Finding:

```
{
 "finding_id": "fnd-001",
 "created_at": "2026-10-28T10:25:02Z",
 "category": "repeated_authentication_failure",
 "severity": "medium",
 "confidence": "rule_match",
 "affected_asset": "demo-host-01",
 "evidence_refs": ["evt-001"],
 "explanation": "Six failures were recorded in the configured time window.",
 "recommended_action": "Review account and source context before taking action.",
```

```
  "status": "open",
  "rule_version": "auth-failure-count-v1"
}
```

Treat example fields as a starting contract, not a universal schema. Timestamps should include timezone information. “Confidence” should not be a probability unless calibrated. Each finding should link back to supporting evidence and a rule/model version.

8.3 Example API contracts

Route	Purpose	Contract notes
GET /health	Confirm app/service is running	Return service status and optional DB readiness separately
POST /events/import	Import a supplied dataset	Validate type, size, schema, encoding; return accepted/rejected counts and warnings
GET /findings	Paginated/filterable finding list	Stable pagination, severity/status filters, no unbounded result set
GET /findings/{id}	Inspect a finding	Include evidence references, rationale, version and timeline
POST /findings/{id}/actions	Record an action/status change	Validate allowed transitions; record actor/time/action/outcome; dangerous actions need confirmation and authorization

Use consistent error objects, such as {"error":"invalid_input","message":"Required column 'timestamp' is missing."}. Do not expose stack traces or secrets to the UI.

8.4 Minimum screens

1. **Overview/status:** data source, last import/run, count of findings, processing errors, and whether data is synthetic/replayed/live.
2. **Findings/cases list:** pagination, status/severity, filters, empty state, loading state, error state.
3. **Finding detail:** evidence, plain-language rationale, calculation/rule, uncertainty, recommended next step, limitation.
4. **Action/history:** what a person or simulation did, when, why, and result.
5. **Import/configuration:** optional if users need it for the core workflow; include safe file constraints.

8.5 Acceptance criteria template

A feature is done only when its behavior is testable. Example for an event analysis MVP: - Given the known sample file, the importer accepts supported rows and reports rejected rows. - Given the fixture labelled “suspicious,” the expected rule creates the expected finding with evidence references. - Given a benign fixture, the rule does not raise the expected alert. - Reimporting the same file does not create uncontrolled duplicate findings. - A user can inspect why the finding exists and record a status/action. - Data persists after restart if persistence is claimed. - A clean startup and demo can be repeated without external network access.

---

9. Rules, ML, and LLM decision framework

```
Is the requirement an explicit policy or known pattern?
 YES -> use deterministic rules first.
 NO  -> Is there a relevant, representative dataset?
        NO -> collect safe examples, use descriptive analysis/rules; no invented model.
        YES -> Are labels available and is classification required?
               YES -> establish baseline, then evaluate supervised ML.
               NO  -> consider statistical/unsupervised analysis for investigation,
                      not a definitive malicious/benign verdict.
```

```
Does the user need natural-language explanation of already-grounded results?
 YES -> templates first; optional LLM only with a verified evidence context and fallback.
 NO  -> do not add an LLM.
```

9.1 Minimum safeguards for model-based output

- Compare to a simple baseline; explain why a model is needed.
- Use a held-out or otherwise defensible evaluation method. Avoid leakage from duplicated events, future data, or target-derived features.
- If labels exist, report precision, recall, false positives, false negatives, and a confusion matrix when appropriate.
- Keep score meaning clear. Anomaly score, confidence, risk severity, and calibrated probability are different things.
- Include evidence and uncertainty in the UI. Do not generate a source event, IOC, or root cause that does not exist in the input.
- For LLMs, retrieve relevant records first, pass only minimal necessary data, treat imported content as untrusted, and require event/evidence references in generated claims.
- Treat model output as advisory, not an automatic authorization/containment instruction.
- If a cloud API is used, disclose the external data path and avoid sending sensitive evidence without permission.

9.2 Examples across domains

- **SOC:** rules correlate repeated failed logins; no ML required if the threshold is the task.
- **Vulnerability prioritization:** transparent weighting of severity, asset exposure, and criticality may be more appropriate than training a classifier without labels.
- **Fraud/anomaly:** statistical deviation may prioritize review, but the result is not proof of fraud.
- **Forensics:** deterministic timestamp normalization and source-linked entity correlation are usually more important than generative summaries.
- **Education:** rubric-based feedback is deterministic; an LLM can help phrase feedback only if the rubric and answer key govern it.

---

10. Security, safety, and responsible testing

- **Authorization and scope:** scan, monitor, test, or modify only explicitly authorized systems. Use provided fixtures or controlled lab targets by default.
- **Input validation:** restrict file type/size, parse safely, reject malformed or unsupported content clearly, and avoid trusting filenames or MIME types alone.
- **Access control:** enforce permissions at the API/service layer, not only by hiding buttons. Test cross-user record access where roles exist.
- **Secrets:** never commit API keys, tokens, passwords, private keys, or actual credentials. Use environment variables and a safe example config.
- **Evidence integrity:** preserve originals or a verifiable copy; record hashes, provenance, collection time, source and tool/version details where relevant. A cryptographic hash can help detect changes to data, but by itself does not prove the original source was authentic or the evidence collection process was valid.
- **Auditability:** record the actor, time, object, reason, and outcome for important actions. Avoid an unbounded log row for every repeated event; deduplicate/group into incidents when appropriate and paginate results.
- **False positives and negatives:** explain them and test both. A match is not proof; no match is not proof of safety.
- **Privacy:** minimize data, redact where possible, use synthetic records for the demo, and avoid sending private data to cloud services without authorization.
- **Safe response:** destructive or disruptive actions should require authorization, explicit confirmation, a clear target/effect, and rollback planning. Simulate containment if live action is unnecessary or unsafe.

- **Uploaded files:** enforce size/time limits, store outside executable/static paths when possible, use generated safe filenames, and do not execute uploads.
- **Cryptography:** use established libraries and protocols; do not invent primitives. Keep secrets out of logs and sample outputs.
- **Responsible demos:** offensive-security demonstration must stay within authorized scope and safe simulation. No destructive malware, unauthorized exploitation, credential theft, or real-world attack instructions.

For a basic reference on security management, the NIST Cybersecurity Framework 2.0 provides high-level outcomes and is intentionally not a prescriptive stack. For web risks, use the current OWASP Top 10 and the OWASP API Security Top 10 as checklists, not as proof of complete security. Links are provided in Section 17.

---

11. Team coordination and parallel development

11.1 Team of 3

Member	Owns	Deliverables	Backup / integration
1. Tech lead + integration/API	Scope, architecture, data contract, backend skeleton	API contract, health route, persistence path, launch instructions	Integrates every few hours; coordinates end-to-end demo
2. UI + workflow	Core screen and user journey	One complete UI flow against agreed sample JSON	Can use mock data briefly; must switch to real endpoint early
3. Analysis + QA	Rules/parsing, fixtures, expected outcomes, test plan	Core logic, unit tests, demo fixture, limitations	Owns correctness tests and helps demo narrative

11.2 Team of 4

Member	Owns	Deliverables	Integration note
1. Tech lead/integration	Scope, contracts, API boundaries, final architecture	Decision log, merge coordination, final pitch	Avoid taking every feature; keep architecture small
2. Frontend	UI workflow	List/detail/action screens	Build against contract, not guessed field names
3. Backend/storage	API validation and database	Import, query, action endpoints, initialization	Write errors and setup docs
4. Security/data + QA	Rules, fixtures, test cases, demo	Expected outcomes, tests, known limitations	Run full vertical slice continuously

11.3 Team of 5

Member	Owns	Deliverables
1. Tech lead/integration	Decision log, contracts, integration, judge pitch	One known-good launch path and final technical narrative
2. Frontend UI	React/Streamlit workflow as selected	Core screens/states/accessibility basics
3. API/database	Request validation, persistence, status actions	Contract-compliant endpoints and migrations/init
4. Analysis engine	Rules, parsers, correlation/scoring	Evidence-linked findings and unit tests
5. QA/demo/security	Fixtures, acceptance tests, limitations, slides/demo	Reproducible demo, bug triage and test evidence

11.4 Team of 6

Member	Owns	Deliverables
1. Tech lead/integration	Scope, system boundaries, merges, final narrative	Architecture decision record and working integrated build
2. Frontend shell/workflow	Navigation and overall state	Working primary user path
3. Frontend data/detail	Tables, evidence detail, empty/error states	Evidence-inspection experience
4. API/database	Contracts, persistence, validation	Stable API and initialization
5. Security analysis/data	Rules, parsing, scoring	Tested core security outcome
6. QA/demo/DevSecOps	Fixtures, security checks, launch/reset, demo	Acceptance results and fallback plan

With larger teams, split only after a shared contract and component boundaries are written down. Do not create additional services just to keep everyone busy.

11.5 Integration rules

- Agree file names, schema, API routes, status values, and error object before parallel coding.
- Build one vertical slice early; integrate working code as soon as each contract exists.
- Keep a stable branch/build and commit runnable checkpoints.
- Limit mock data to the UI's early development; assign an explicit deadline to replace it.
- Assign a single integration owner; all owners must explain their changed files and assumptions.
- Test a clean launch and reset repeatedly, not only the developer's warm local session.
- Resolve API/field mismatches by updating the contract, not by hidden frontend transformations that obscure the source of truth.

---

12. Technical justification for judges

12.1 The nine-question defense

For every major component, answer: 1. Which requirement motivated it? 2. Why does it fit the input, output, and user workflow? 3. Which alternative did we consider? 4. Why was that alternative not as suitable for this time/skill/data constraint? 5. What trade-off did we accept? 6. How did we handle security, privacy, and authorization? 7. What works offline, and what is simulated or optional? 8. How did we test correctness and failure cases? 9. What would need to change for production scale?

12.2 Pitch template

> "We built **[product]** for **[specific user]** to address **[specific risk/problem]**. It takes **[inputs]**, validates and analyzes them using **[rules/model/policy]**, and returns **[specific output]** with **[evidence/rationale]**. We selected **[stack]** because it fits **[data, skill, offline and time constraints]**; we considered **[alternative]** but rejected it because **[trade-off]**. We verified the core path with **[fixtures/tests/metrics]**. The current prototype **[clearly stated limits]**; production use would require **[key next steps]**."

12.3 Example justifications

SOC: "React/FastAPI/SQLite keeps the analyst workflow separate from explainable Python correlation logic and allows the demo to run on local event fixtures. We chose rules over ML because the available time and expected labels did not justify model training. We test duplicate handling, expected alerts, false positives, evidence inspection, and status history. The sample feed is replayed, not a production sensor."

Forensics: "Python and SQLite support local parsing and source-linked timelines without external services. We preserve source references and hash evidence to help detect subsequent changes; we do not claim that a hash alone proves provenance. Known sample timestamps and event relationships are checked by tests."

Vulnerability management: “A relational store makes assets, findings, remediation owners, and verification status easy to join and audit. We use supplied scan findings rather than claim comprehensive scanning. A finding is marked verified only after a documented retest or explicit verification evidence.”

Cloud configuration: “Snapshot-based JSON/YAML policy checks let us demonstrate deterministic findings without cloud credentials. Each result names the rule and resource path. Unknown fields are not silently treated as passing, and the prototype does not claim certification.”

Education: “Scenario-based scoring and rubric feedback are deterministic, reproducible, and protect the core learning workflow from internet dependencies. Teacher/student views use distinct permissions and synthetic learner records. Scores indicate performance on this exercise, not proof of real-world security competence.”

AI/ML: “We compare the proposed model with a simple baseline on a defensible held-out set. We report error trade-offs rather than claiming AI improves detection automatically. Generated summaries cite finding IDs and do not create evidence; the core outcome still works if the model/API is unavailable.”

---

13. Failure modes and fallback plans

Failure / warning sign	Immediate response	Reduced-scope fallback	Minimum acceptable outcome / demo disclosure
React UI is behind schedule	Freeze styling; finish the primary path only	Streamlit or a single functional page if allowed by architecture	State which UI elements are incomplete; prove analysis and evidence work
Database service fails	Check connection config; preserve fixture; do not silently ignore writes	SQLite/local fixture when schema supports it, or read-only report mode	Disclose persistence limitation; do not claim durable records if unavailable
Internet disappears	Switch off external dependencies and API calls	Local fixtures, templates, local DB	Clearly label offline mode and the age/source of bundled data
API integration breaks	Inspect request/response schema and errors; test with one known request	Temporarily run analysis in one process or return a stable mocked fixture clearly labelled	Do not present mock output as integrated live analysis
Dataset too small	Limit claims; check whether deterministic rules are adequate	Rule-based baseline or scenario demo	State sample size and evaluation limits; do not claim generalization
ML performs poorly	Compare to baseline and inspect errors	Disable ML and show transparent rule/statistical results	Show measured result honestly; don’t cherry-pick without disclosure
Threat feed unavailable	Keep local feed and its timestamp	Indicator matching against fixture	Disclose that enrichment is bundled, dated, or absent
Dependency install fails	Check lock/installed packages; remove optional dependency	Standard library, existing tools, or simpler parser	Report the feature removed; preserve runnable core
Live monitoring fails	Test replay path	Preloaded event/PCAP/flow replay	Call it a replay/simulation, not live collection

Failure / warning sign	Immediate response	Reduced-scope fallback	Minimum acceptable outcome / demo disclosure
Too many planned features	Return to acceptance criteria and cut list	One vertical slice, 3-5 criteria	Demonstrate one coherent workflow rather than a collection of screens
Dashboard looks good but analysis doesn't work	Prioritize core rule/function and fixtures immediately	Simple list/table with transparent results	Visual polish does not substitute for a working security outcome
Late major requirement appears	Map it to input/output and acceptance criteria; assess trade-off	Implement the smallest valid slice or document as limitation	Explain what is implemented and what remains; don't make hidden assumptions
Duplicate alerts/log growth	Add stable dedupe key, grouping, pagination/limits	Group to one incident and keep underlying event count/reference	State dedupe policy and verify one event/incident does not create unbounded records
State/count doesn't reset	Define reset scope and single state source	Reload from database or known clean fixture	Show a clean demo reset and document what is preserved
Model/LLM returns ungrounded claim	Hide or flag claim, verify evidence references	Deterministic templates/rules	Never display generated text as verified evidence

14. Testing and demonstration plan

14.1 Test matrix

Test class	Test cases	Expected proof
Unit	Rule match, no-match, boundary threshold, timezone, normalization	Deterministic outcome and clear reason
Input	Empty, malformed, missing columns/fields, unexpected type, oversized file	Safe error/warning; no app crash or silent data loss
Data quality	Duplicate events, conflicting timestamp formats, unknown fields	Documented dedupe and normalization behavior
API	Valid request, invalid schema, missing record, invalid state transition	Stable status/error contract; safe messages
Persistence	Import -> restart -> query finding; action survives restart	Durable state if persistence is claimed
Security	Unauthorized record access, upload path, secret leakage, error output	Boundaries enforced and sensitive information excluded
Model	Positive/negative fixture, held-out evaluation where possible	Metrics appropriate to task; limitations explained
Integration	Full vertical slice from input to audited outcome	One repeatable success path
Demo reliability	Clean startup, DB initialization, reset, no network	Judges can follow without developer-only hidden state

14.2 Metrics that match the product

- **Detection with labels:** precision, recall, false-positive rate, false-negative count, and confusion matrix when meaningful.
- **Incident response:** time from input to inspectable finding, deduplication rate on known duplicates, percent of findings with evidence references, and workflow completion.
- **Forensics:** source-reference coverage, timestamp normalization checks, reproducibility on the same evidence, and parser error visibility.
- **Vulnerability management:** deduplication of findings, time to assign/triage, verified-vs-unverified closure count.
- **Configuration analysis:** number of known failing/passing fixtures handled correctly and percentage of findings with resource/rule references.
- **Education:** answer/rubric consistency, completion, repeat attempt improvement; avoid treating one quiz score as a complete measure of competence.
- **Reliability:** clean launch success, persistence across restart, expected behavior on malformed/empty input.

A passing software test proves behavior for that test; it does not prove production-grade security. State sample size, scope, fixtures, known blind spots, and whether operations are simulated.

14.3 Five-minute demo narrative

1. **Problem (30 sec):** target user and real-world friction.
2. **Input (30 sec):** show sample/source and clearly state if synthetic or replayed.
3. **Core analysis (60 sec):** execute the primary workflow and explain rule/model/policy logic.
4. **Evidence (60 sec):** open one finding, show source references, reason, severity/confidence meaning, and uncertainty.
5. **Action (45 sec):** record a safe action/status and demonstrate audit history.
6. **Verification (45 sec):** show known expected result or test evidence, dedupe/restart check if relevant.
7. **Impact and limits (30 sec):** connect to success criterion and explain what would be required for production use.

---

15. Copy-ready decision templates

Keep these concise enough to complete on paper. Use only materials allowed by the event.

A. Problem-statement analysis

```
Target user:
Asset / decision being protected:
Observed pain or threat:
Required outcome:
Inputs available:
Required action/output:
Constraints / judging criteria:
Assumptions (explicit):
Out of scope:
```

B. Domain classification

```
Primary domain:
Secondary domain (if essential):
Why this classification fits:
Core user decision:
Data source: supplied / synthetic / local / authorized live
Needs live telemetry? Why?
```

C. Architecture selection

UI/interface:
Backend/API (or none):
Storage:
Analysis/policy method:
Live update method (or none):
Offline path:
One-sentence reason for each component:
Fallback architecture:

D. Technology comparison

Decision to make:
Option A + benefit + cost:
Option B + benefit + cost:
Team familiarity:
Offline/setup risk:
Final choice and requirement it satisfies:

E. MVP scope

Primary user story:
Vertical slice, step 1 -> step N:
Acceptance criterion 1:
Acceptance criterion 2:
Acceptance criterion 3:
Must-have:
Explicitly deferred:
Demo success condition:

F. Data contract

Object name:
Required fields/types:
Optional fields:
Stable ID / dedupe key:
Timestamp format/timezone:
Evidence/source reference:
Success response shape:
Error response shape:
Persistence behavior:

G. Team responsibility

Member -> owned files/component -> deliverable -> dependencies
Integration owner:
QA owner:
Demo/presentation owner:
First integration checkpoint:

H. Technical risk register

Risk / failure mode:
Likelihood (low/medium/high):
Impact (low/medium/high):
Early warning sign:
Fallback:
Owner:
What must be disclosed:

I. Testing plan

Known benign fixture and expected result:
Known suspicious/positive fixture and expected result:
Malformed/empty test:
Duplicate/restart test:
Security boundary test:
End-to-end demo path:
Metric and limitation:

J. Judge-facing technology justification

```
Requirement -> chosen technology -> why it fits
Alternative considered -> why rejected
Security/privacy consideration:
Offline fallback:
Test evidence:
Known limitation:
Production next step:
```

---

16. Pocket reference sheet

Default stack

React + TypeScript + Tailwind/Vite | Python + FastAPI | SQLite | transparent rules/Pandas | pytest + local fixtures.

Fast analytics

Streamlit + Python + Pandas + SQLite. Use this when data analysis, tables, or investigation are the core value.

Offline utility

Python CLI or Streamlit + SQLite/local files + preloaded fixture. Do not make cloud/API availability the only way the demo works.

Domain-to-stack map

- SOC/incident response: React + FastAPI + SQLite/PostgreSQL + rules/correlation.
- Forensics/evidence: Python + parser/Pandas + SQLite + local source references; graph library only if needed.
- Vulnerability/configuration: parsers + explicit rules + asset/finding store + report UI.
- Web/API security: local fixtures, safe tests, pytest/HTTPX or existing stack tests.
- Network detection: prepared PCAP/flows + Python/Scapy if installed + rules.
- Identity/Zero Trust: explicit policy function + audit log + synthetic identities.
- Privacy: local processing + structured detectors + redaction review.
- Education: scenarios + deterministic rubric + learner/teacher separation.
- AI/ML: baseline and evaluation first; optional model only with appropriate data.

Storage rules

- SQLite: one-machine MVP and durable local records.
- PostgreSQL: concurrent users, relational joins and constraints justify setup.
- MongoDB: document-shaped records/query patterns deliberately fit.
- JSON/CSV: fixtures/import/export; not a full shared case-management database.
- In-memory: throwaway demo state only.

Rules vs ML vs LLM

- Explicit known condition -> rule.
- Data pattern without labels -> descriptive/statistical investigation, not verdict.
- Representative labelled data + baseline + evaluation -> consider supervised ML.
- Natural-language explanation of known findings -> templates first; optional grounded LLM.
- No evidence/model justification -> no model.

30-minute flow

0-4 problem -> 4-7 domain -> 7-10 inputs/outputs -> 10-14 workflow -> 14-18 stack -> 18-21 MVP -> 21-24 contract -> 24-27 owners/risks -> 27-30 pitch/freeze.

Top 10 anti-patterns

1. Dashboard before analysis.
2. More than one primary workflow.
3. ML/LLM with no data or evaluation.
4. Only-online demo path.
5. Microservices or infrastructure with no requirement.
6. Unknown or inconsistent data contracts.
7. Fake “live” monitoring using unlabeled replay.
8. Unbounded duplicate logs/alerts.
9. Unsupported certainty about evidence, score, or remediation.
10. Testing only the happy path or demoing an unintegrated build.

Final architecture approval checklist

- ☐ One problem and one user workflow.
- ☐ One primary analysis method.
- ☐ Every component has a requirement-based reason.
- ☐ Data contract, source reference, and error behavior are agreed.
- ☐ Works offline or has an honest fallback.
- ☐ Fixtures and expected outcomes exist.
- ☐ Duplicate, malformed, empty, and persistence behavior are covered where relevant.
- ☐ Safe testing/authorization boundaries are explicit.
- ☐ One known-good startup and reset path.
- ☐ Demo labels synthetic/replayed/live data accurately.

---

17. Technical accuracy and reference sources

This playbook avoids asserting exact library version numbers because a pinned version depends on the environment available on the day. Verify package versions and compatibility when web access and event rules permit, before implementation or production deployment. The following official/primary sources are useful for post-ideation verification:

- **Python:** https://docs.python.org/3/
- **FastAPI:** https://fastapi.tiangolo.com/
- **React:** https://react.dev/
- **TypeScript:** https://www.typescriptlang.org/docs/
- **Vite:** https://vite.dev/guide/
- **Tailwind CSS:** https://tailwindcss.com/docs
- **Streamlit:** https://docs.streamlit.io/
- **Gradio:** https://www.gradio.app/docs
- **SQLite:** https://www.sqlite.org/docs.html
- **PostgreSQL:** https://www.postgresql.org/docs/
- **MongoDB:** https://www.mongodb.com/docs/
- **Pandas:** https://pandas.pydata.org/docs/
- **NumPy:** https://numpy.org/doc/
- **scikit-learn:** https://scikit-learn.org/stable/user_guide.html
- **NetworkX:** https://networkx.org/documentation/stable/

- **Scapy:** https://scapy.readthedocs.io/
- **pytest:** https://docs.pytest.org/
- **HTTPX:** https://www.python-httpx.org/
- **NIST Cybersecurity Framework 2.0:** https://www.nist.gov/cyberframework
- **OWASP Top 10 (2025):** https://owasp.org/www-project-top-ten/
- **OWASP API Security Top 10 (2023 edition):** https://owasp.org/www-project-api-security/
- **MITRE ATT&CK:** https://attack.mitre.org/

The NIST CSF is an outcome-oriented risk framework, not a prescribed technology stack. OWASP lists are awareness and review aids, not a certification of security. MITRE ATT&CK is a knowledge base for adversary tactics and techniques; mapping a finding to a technique does not itself prove that activity occurred.

Source-derived context used in this edition

The original *Cybersecurity Hackathon - Tech-Lead Field Guide* supplied the default stack, fast analytical fallback, 30-minute decision sequence, domain-to-stack mappings, evidence/data contracts, responsible-demo rules, common anti-patterns, metrics, team coordination advice, and judge pitch structure. This edition expands those ideas into explicit selection matrices, additional domains, architecture blueprints, fallback behavior, test cases, role splits, and reusable templates rather than discarding the original recommendations.

The supplied *HackMITTEN 3.0 Participant Instructions* specifies the event's first-round device/AI restriction, writing on the organizer-provided KG sheet, the 28-29 October 2026 event context, development/testing/demo phases, team ownership, originality requirements, and safety rules. Follow the organizers' latest announcements if they revise timing or procedures.

---

18. Final technical-lead operating principles

1. Solve the given problem, not the problem you hoped to receive.
2. Fit the stack to the data, user, timeline, team skills, and permitted environment.
3. Prefer explainable analysis with evidence over unexplained scores.
4. Keep the core workflow operational without optional services.
5. Make uncertainty, simulation, assumptions, and limitations visible.
6. Test the analysis and contracts before spending time on styling.
7. Group and bound events; keep audit history useful and readable.
8. Use safe fixtures and authorized systems only.
9. Integrate early, rehearse a clean startup, and maintain one fallback path.
10. Never claim a control, detection, remediation, model, or evidence guarantee that the prototype has not verified.

Final rule: One problem. One primary workflow. One core analysis method. One reliable demo path.